ConsensusCode-Evolve: Token-Efficient Generative Code Search for Consensus-Aware Multi-Agent Inventory Automation

Author Name

***Abstract*—Generative AI is creating new opportunities for industrial automation, but direct deployment of large language models (LLMs) as online controllers remains difficult because of cost, safety, interpretability, and reproducibility concerns. This paper proposes *ConsensusCode-Evolve*, a simulator-grounded framework that uses an LLM as an offline code-evolution engine for multi-agent inventory automation. The target problem is multi-site replenishment and transshipment, a representative discrete manufacturing and supply-operation task in which local inventory agents must coordinate material movements under demand uncertainty, lead times, capacity limits, and asymmetric cost structures. The paper also formulates a simulator benchmark for this setting, since existing studies cover related inventory and supply-chain control problems but do not provide a widely used benchmark for the full setting considered here. Instead of asking an LLM to control the system online, the proposed method asks it to generate constrained Python policy functions for replenishment, transshipment proposal, and consensus gating. Candidate policies are validated by an abstract-syntax-tree safety checker, evaluated in the simulator, and selected through an archive-based loop using compact numerical feedback. A consensus-seeking layer models donor willingness, receiver urgency, and fairness debt, so transshipment actions are accepted only when they satisfy both local and system-level requirements. The resulting policies are lightweight, executable without an LLM at deployment time, and directly inspectable by engineers. The paper outlines the benchmark problem, simulator, method, safety constraints, evaluation protocol, and planned comparisons against genetic programming, heuristic, and learning-based baselines. The central hypothesis is that simulator-grounded generative code search can discover interpretable, consensus-aware inventory policies with lower token cost than online LLM agents while retaining automation-oriented robustness and deployability.**

***Index Terms*—Generative AI, industrial automation, large language models, multi-agent systems, consensus seeking, inventory control, transshipment, code generation, simulation optimization.**

I. Introduction

Industrial automation increasingly requires decision-making systems that coordinate distributed assets, adapt to uncertain operating conditions, and remain interpretable to engineers. In discrete manufacturing and supply operations, material-flow decisions such as replenishment, inventory allocation, and transshipment are not merely numerical optimization tasks. They involve multiple sites or production units with local constraints, asymmetric costs, and sometimes conflicting interests. A site with surplus inventory may be asked to support another site, but such a transfer can be harmful if the donor later faces a demand surge. This makes multi-site inventory automation a natural setting for studying distributed decision-making and consensus-aware control.

Similar decision settings arise in many supply chains. Large retailers must balance stock across stores, regional warehouses, and distribution centers. E-commerce firms operate fulfillment networks where local shortages and overstock can occur at the same time. Manufacturers coordinate raw materials, components, and finished goods across plants. Spare-parts networks in transportation, energy, and equipment maintenance must avoid service failures while limiting slow-moving inventory. Pharmaceutical, medical, grocery, and fresh-product supply chains face a further tension: shortages can be costly, but excess stock can also lead to waste, expiration, or high holding cost. These examples differ in detail, but they share the same core difficulty: local inventory decisions affect the whole network.

Large language models (LLMs) create a new way to generate decision logic, but direct online use of an LLM as an industrial controller is difficult. Online calls add cost and latency. They also make reproducibility, safety review, and engineering inspection harder. Inventory control adds further constraints: decisions must respect material availability, capacity, lead times, order costs, and transfer costs. A useful generative method for this setting should therefore be grounded in the simulator or plant model, should produce deterministic policies for deployment, and should expose the decision logic in a form that engineers can inspect.

Recent progress in LLM-based code generation and program search suggests one promising path. AlphaCode showed that large language models can generate many candidate programs and use selection mechanisms to solve competitive programming problems [1]. CodeRL incorporated execution feedback into code generation [2]. Self-Refine and Reflexion demonstrated that language models can iteratively improve outputs using compact feedback or verbal memory [3], [4]. Voyager used LLM-generated executable code as reusable skills in an embodied environment [5], and Eureka showed that LLMs can write reward functions evaluated through simulation [6]. FunSearch is particularly relevant because it combines LLM-generated programs with automatic evaluation to discover high-performing mathematical constructions [7]. These studies suggest that an LLM need not be the controller itself; instead, it can serve as a generator of candidate executable programs, while an external evaluator provides grounding.

This paper transfers that idea to industrial inventory automation. We propose *ConsensusCode-Evolve*, a token-efficient

framework in which an LLM evolves small, safe Python policy functions offline. Each candidate policy contains three functions: a replenishment policy, a transshipment proposal policy, and a consensus gate. The replenishment function determines local order quantities. The transshipment proposal function suggests pairwise material movements. The consensus gate accepts, scales, or rejects proposed transfers based on donor willingness, receiver urgency, network benefit, and fairness debt. Candidate code is statically checked before execution and then evaluated in a multi-site inventory simulator. Only compact performance summaries and selected policy code snippets are returned to the LLM for the next mutation round.

The proposed approach differs from direct LLM planning in three ways. First, the LLM is used offline, which reduces token cost and avoids dependence on a large model during deployment. Second, generated code is constrained to a small safe subset of Python, making the final policies inspectable and reproducible. Third, the method embeds autonomous multi-agent consensus seeking into the action layer, so transshipment decisions are not treated as forced centralized commands but as negotiated actions among local inventory agents.

The intended contributions are as follows:

- A configurable simulator benchmark for multi-site replenishment and transshipment with local site states, demand forecasts, capacity limits, cost asymmetry, and pairwise transfer decisions.
- A simulator-grounded generative code-evolution framework for multi-site replenishment and transshipment automation.
- A consensus-aware policy representation that separates replenishment, transshipment proposal, and agreement gating.
- A lightweight safety mechanism based on abstract syntax tree validation for generated industrial policy code.
- An evaluation protocol comparing LLM-evolved policies with genetic programming, heuristic, and learning-based baselines under cost, service, fairness, and robustness metrics.

The claims in this paper are limited to the method, the implemented prototype, and the evaluation design. Empirical superiority over baselines should be claimed only after the full experiments in Sections V and VI are completed.

II. Background and Related Work

A. Multi-Site Inventory and Material-Flow Automation

The problem studied in this paper belongs to multi-location inventory control and material-flow automation. A set of sites, such as plants, warehouses, production cells, or spare-parts depots, must decide when to order inventory and how to share inventory across locations. The basic inventory tradeoff is well known: high inventory reduces lost sales or service failures, but it increases holding cost and may waste capacity; low inventory saves capital, but it raises the risk of shortage. Classical inventory texts describe this tradeoff through reorder-point, base-stock, and (s, S) policies under demand uncertainty and replenishment lead times [8], [9].

Multi-site systems are harder than single-site systems for three reasons. First, each site has a local state, but the value of an action depends on the network state. A site with high inventory may be a good donor when the rest of the network is short, but the same site may need to keep stock when its own forecast is high. Second, replenishment and transshipment decisions interact. Ordering too much can reduce future shortages but may also make transshipment unnecessary and costly. Transshipping too aggressively can lower current lost sales but create future donor shortages. Third, industrial automation must respect physical and operational constraints, including lead times, capacity, order costs, and transfer costs. These constraints make the control problem a sequential decision problem rather than a one-period allocation problem.

This paper focuses on replenishment and lateral transshipment. Replenishment brings new material into the network after a lead time. Lateral transshipment moves inventory between sites at the same echelon. Reviews of lateral transshipment models show that pooling inventory can improve service and reduce total stock, but the decision rules become difficult when demand is stochastic, costs are asymmetric, and sites differ in capacity or service pressure [10]. Prior work on supply-chain inventory review and control also shows that inventory decisions must be modeled as dynamic systems with uncertain demand and coupled decisions [11], [12]. In industrial automation, this difficulty is not only computational. A transfer also changes the risk exposure of the donor site. This motivates a policy representation that treats transshipment as an agreement between local agents rather than as a forced movement from a central planner.

B. Production-System Modeling and Scheduling

A second body of work studies how production and logistics systems should be modeled before decisions are optimized. Manufacturing systems are often discrete, constrained, and history dependent. Petri-net models have been used to describe production sequences and resource constraints [13]. Systems approaches to batch-discrete manufacturing model production stages, buffers, and improvement actions at the system level [14]. Stochastic continuum models have also been used for control of multi-line re-entrant manufacturing plants [15]. These studies show that industrial decisions should be grounded in the physical and operational structure of the system.

Scheduling research gives a related view. In semiconductor and job-shop systems, reinforcement learning has been used for robust scheduling and real-time dynamic scheduling under changing conditions [16], [17]. Cloud manufacturing studies also use deep reinforcement learning to assign tasks when rewards are sparse and resources are distributed [18]. These methods can react to complex system states and learn from simulation, but they usually produce scheduling policies rather than inspectable inventory-transfer code. They also do not address the donor-risk and fairness issues that appear when one site gives inventory to another. The present paper adopts the same need for simulator-grounded decisions, but studies replenishment and transshipment policies whose final form is executable code.

C. Analytical and Rule-Based Inventory Policies

The first category of methods uses analytical or rule-based policies. Representative examples include reorder-point policies, order-up-to policies, and (s, S) policies. These methods are attractive because they are simple, fast, and easy for engineers to inspect. They also give strong baselines for inventory problems with regular demand and relatively simple cost structures [8], [9].

Their limit is that hand-designed rules often encode only a small part of the system state. A local (s, S) rule may handle replenishment well for one site, but it does not naturally decide how much inventory should be moved between sites under asymmetric holding costs, shortage costs, and forecasts. Rule-based lateral transshipment policies can be added, for example by moving inventory from surplus sites to shortage sites. However, such rules often require fixed thresholds and may not capture donor risk, receiver urgency, and repeated one-way support in the same decision. In the present paper, these policies are useful as interpretable baselines and as seed policies for generative code search, but they do not provide an automatic way to design consensus-aware transfer logic.

D. Optimization, Simulation, and Digital-Twin-Based Search

The second category formulates inventory control as an optimization problem. Exact dynamic programming can be used for small systems, but the state and action spaces grow quickly with the number of sites, inventory levels, pipeline orders, and transfer decisions. For larger stochastic systems, simulation-based optimization is often used: a candidate policy is evaluated in a simulator, and a search method improves policy parameters or policy structure based on simulated performance [19], [20].

Simulation-based search fits the problem in this paper because the simulator can include lead times, lost sales, capacity, order costs, and lateral transfers directly. It also supports fair comparison among very different policy classes. Related studies use digital twins and data-driven models to support scheduling and multi-objective optimization in manufacturing systems [21], [22]. Such work is valuable because it connects decision learning to a model of the plant or process. The weakness is that simulation feedback can be expensive and noisy. A search method must decide how to use limited evaluation budget. If the policy class is too simple, the search cannot express useful coordination behavior. If the policy class is too large, many candidates are invalid, unsafe, or hard to interpret. ConsensusCode-Evolve follows the simulation-based view, but it uses an LLM to propose small executable policy programs and uses the simulator only as the judge.

E. Evolutionary and Genetic Programming Policies

The third category learns symbolic decision rules through evolutionary computation. Genetic programming (GP) evolves expression trees rather than numeric parameters [23], [24]. This is useful for inventory automation because a GP tree can map state variables, such as inventory, forecast, capacity, and cost, to an order or transfer quantity. Multi-tree GP can represent different decisions with separate trees, for example one tree for replenishment and another tree for transshipment. Cooperative coevolution can further split a large policy into interacting components.

The strength of GP is that the learned rules are executable and more inspectable than many black-box policies. GP can also discover nonlinear combinations of state variables without requiring a differentiable simulator. These features make GP a strong baseline for this project. The limit is that GP search is usually driven by syntactic mutation and crossover. It may need many evaluations to discover higher-level program structure, such as a donor-protection rule followed by a fairness correction. GP trees also tend to express decisions as direct mappings from state to action. They do not naturally separate a transshipment proposal from a later agreement gate unless this structure is built into the representation.

ConsensusCode-Evolve keeps the useful idea of executable symbolic policies, but replaces random program edits with LLM-generated code mutations constrained by an explicit API. This does not remove the need for simulation evaluation. Instead, it changes the source of candidate programs while keeping the simulator as the authority on performance and feasibility.

F. Learning-Based Inventory Control

The fourth category uses reinforcement learning (RL) and deep RL to learn policies through interaction with a simulated environment. RL is a natural fit for sequential inventory problems because actions affect future states, and the objective is cumulative cost or reward [25]. Deep RL methods, including value-based and policy-gradient methods, can represent complex policies in large state spaces [26], [27]. In inventory systems, RL can learn from simulated demand trajectories and can adapt to nonlinear interactions among replenishment, shortages, and transfers.

The main strength of RL is flexibility. It can use rich state information and optimize long-run reward directly. Its main limits for the present paper are deployment trust and interpretability. A neural policy can be difficult to inspect, and small changes in scenario design may require retraining or careful validation. In addition, direct RL does not by itself explain whether a transfer is fair to the donor or acceptable to both sites. These issues are important in automation settings where engineers must understand why a material movement is recommended. For this reason, RL is best treated here as a learning-based baseline rather than as the central design model.

G. Multi-Agent Coordination and Consensus Seeking

The fifth category studies distributed decision-making and consensus among agents. Consensus algorithms are widely used in multi-agent control to align decisions or estimates through local interaction [28]. Agent-based scheduling has also been studied through bidding languages that let agents express requirements during scheduling [29]. Human-machine collaborative systems in intelligent manufacturing further show that industrial decision support must be understandable to people who review or modify the system output [30]. In supply

and inventory settings, the same idea appears in a practical form: a local unit may not want to give up inventory unless the transfer is justified by system benefit and does not create unacceptable local risk. Thus, the decision is not only "how much should move?" but also "should the donor agree?" and "is the repeated pattern of support acceptable?"

This view is closely related to lateral transshipment. A centralized model can optimize all transfers from a network perspective, but industrial operations often contain local ownership, local service targets, and different risk exposure. A consensus-aware policy can make these tensions explicit. In this paper, the consensus gate is not a general communication protocol. It is a small executable function that accepts, scales, or rejects a proposed transfer using donor state, receiver state, and transfer history. This choice is deliberately modest. It gives the policy a place to encode donor willingness, receiver urgency, and fairness debt without claiming that a full negotiation protocol has been solved.

H. LLM Agents, Code Generation, and Program Search

The final category is generative AI for executable decision logic. Recent work shows that LLMs can generate code, improve outputs using feedback, and participate in program search. AlphaCode generated and filtered many candidate programs for competitive programming [1]. CodeRL used execution feedback to improve code generation [2]. Self-Refine and Reflexion showed that compact feedback can guide iterative improvement [3], [4]. Voyager used generated code as reusable skills in an embodied environment [5]. Eureka used LLM-generated reward code evaluated in simulation [6]. FunSearch combined LLM-generated programs with automatic evaluation to find high-performing constructions [7]. Related optimizer-style prompting methods also show that LLM outputs can be improved through iterative candidate generation and scoring [31], [32].

These studies suggest a useful direction for industrial automation: the LLM does not need to act as an online controller. It can instead propose candidate programs offline, while a task simulator checks whether those programs work. This separation is important for inventory automation because online LLM calls are costly, hard to reproduce exactly, and difficult to certify. However, existing LLM code-generation work is usually not designed around inventory physics, lateral transshipment, or local agreement among industrial agents. It also often assumes that generated programs are judged by unit tests or benchmark scores, whereas this paper judges policy code by stochastic simulator performance and operational metrics.

I. Supply-Chain Control Benchmarks

Benchmark environments are important because they make it possible to compare methods under the same dynamics, cost model, and information structure. In supply-chain control, the Beer Game has long been used to study ordering behavior, delayed information, and the bullwhip effect; it has also been used as a testbed for reinforcement learning in ordering decisions [33]. OR-Gym provides reinforcement-learning environments for several operations research problems, including inventory management, and has helped make OR problems easier to test with standard RL interfaces [34]. Recent studies also examine multi-echelon or multi-location inventory control with deep reinforcement learning and multi-agent reinforcement learning [35], [36].

These benchmarks and models are useful, but they do not directly cover the full setting studied here. Beer Game environments focus on ordering and information delay in a serial supply chain. Multi-echelon inventory environments focus on upstream-downstream replenishment and allocation. Multi-location studies address distributed inventory, but many do not include explicit pairwise lateral transshipment with donor feasibility, local agreement logic, forecast features, and fairness-oriented coordination metrics. To the best of our knowledge, there is no widely used benchmark that directly models the multi-site, multi-agent replenish-and-transship setting considered in this paper. Existing studies cover related parts of the problem, but they do not provide a standard benchmark for comparing policies that jointly decide replenishment, transshipment proposal, and consensus-aware transfer acceptance.

The related work covers methods with different assumptions about the decision maker, the policy form, and the evaluation setting. A category-level comparison is therefore useful before stating the research gap. Table I summarizes the main categories discussed above and lists representative studies for each category. The table does not rank the methods. It shows which parts of the present benchmark setting are naturally supported by each category and which parts usually require extra modeling choices.

The main takeaway from Table I is that existing categories cover important parts of the problem. Classical and multi-echelon inventory models give clear replenishment logic. Stochastic, robust, and simulation-based methods support uncertainty and detailed evaluation. RL and multi-agent methods support sequential or distributed decisions. Forecast-driven methods connect control to demand information, and code-generation methods show how executable programs can be produced. What is less common is the combination needed here: multi-site inventory, local agent decisions, multiple forecast inputs, executable policy logic, lateral transshipment agreement, and a common replenish-and-transship simulator. The proposed method is positioned in this gap. It does not claim to replace these categories or to be better in all settings. It uses the simulator to define the setting and to evaluate generated policy code under the same transition, feasibility, and cost rules.

J. Research Gap

The literature above leaves a clear gap. Classical policies are simple and inspectable, but they have limited ability to express adaptive consensus-aware transshipment. Optimization and simulation-based methods can evaluate realistic systems, but they need a policy representation that is expressive and still safe. GP provides executable symbolic rules, but its edits

TABLE I
CATEGORY-LEVEL COMPARISON OF INVENTORY-CONTROL AND DECISION-GENERATION METHODS

Method/category	Representative studies	Main idea	Strengths and limitations	Relation to this problem	MS	MA	KF	Exec.	BE
Classical inventory control	[8]–[10]	Fixed rules such as base-stock, reorder-point, (s, S), or lateral-transshipment rules	Simple and inspectable; usually needs hand tuning and limited state coupling	Strong baseline for replenishment; consensus-aware transfer logic must be added separately	Part.	No	Part.	Yes	No
Multi-echelon inventory optimization	[9], [11], [37]	Optimize ordering and allocation across upstream and downstream stages	Models network structure and lead times; state space can grow quickly	Related to supply networks, but different from same-echelon lateral transshipment	Yes	Part.	Part.	Part.	No
Stochastic or robust optimization	[12], [15], [38]	Optimize decisions under stochastic dynamics or uncertainty sets	Gives explicit uncertainty treatment; often needs tractable approximations	Useful for uncertainty modeling, but less direct for executable consensus gates	Part.	No	Part.	Part.	No
Simulation-based optimization and digital twins	[19]–[22]	Evaluate candidate policies in a simulator or digital twin	Can include detailed transitions and costs; evaluation budget matters	Provides the evaluation principle used in this paper	Yes	Part.	Yes	Part.	Yes
Genetic programming policies	[23], [24]	Evolve symbolic decision rules or trees	Produces executable rules; search edits may need many evaluations	Strong baseline for replenishment and transshipment rules in this codebase	Yes	Part.	Yes	Yes	Part.
RL for inventory or production control	[16], [25], [27], [33]	Learn policies through simulator interaction	Flexible for sequential decisions; policies can be hard to inspect	Learning-based baseline when the simulator action space matches	Yes	No	Yes	No	Part.
Multi-agent RL or consensus methods	[28], [29], [35]	Learn or design distributed decisions among agents	Represents local agents; training and interpretation can be difficult	Related to donor risk, receiver need, and agreement in transshipment	Yes	Yes	Part.	No	Part.
Forecast-driven inventory methods	[11], [36], [39]	Use forecast features to guide ordering, allocation, or learned control	Uses predictive demand information; does not by itself define coordination	Motivates the forecast vector in the policy state	Part.	No	Yes	Part.	Part.
LLM or code-generation methods	[1], [2], [6], [7]	Generate candidate programs, reward code, or policy code	Can propose structured code; needs validation and task grounding	Supplies the generative search idea adapted here	Part.	Part.	Part.	Yes	Part.
Supply-chain control benchmarks	[33]–[36]	Provide reusable environments or common settings for comparison	Enables repeatable tests; existing settings often focus on ordering or echelon flows	Motivates a benchmark for same-echelon replenish-and-transship decisions	Yes	Part.	Part.	Part.	Yes
ConsensusCode-Evolve	This work	Offline LLM-based search over safe policy code with replenishment, transshipment proposal, and consensus gating	Uses executable decision logic; claims depend on the planned experiments and scenarios	Directly targets the benchmark setting studied in this paper	Yes	Yes	Yes	Yes	Yes

MS: multi-site inventory. MA: multi-agent decisions. KF: multiple demand forecast features. Exec.: executable decision logic. BE: benchmark-style evaluation. "Part." means the category can support the item in some formulations, but it is not a default feature of the category.

are mostly syntactic and may struggle to create structured agreement logic efficiently. RL can learn complex behavior, but the resulting policies are often hard to inspect and may not expose donor risk or fairness. Existing supply-chain benchmarks support important related problems, but they do not provide a common benchmark for the full multi-site, multi-agent inventory setting studied here. LLM program search can generate meaningful code, but it has not been adapted to safe, simulator-grounded, multi-site inventory automation with explicit transshipment consensus.

This paper addresses that gap by combining a benchmark simulator, constrained executable policy code, simulator-based evaluation, and a consensus gate for lateral transshipment. The result is an offline generative code-search method whose deployed policy is a small Python program, not an online LLM agent.

III. BENCHMARK PROBLEM AND SIMULATOR

This paper studies a replenish-and-transship benchmark problem for multi-site inventory management. The system contains several inventory sites that face uncertain, time-varying demand. Each site is treated as a local inventory agent with its own stock level, cost parameters, capacity, pipeline information, and demand forecasts. At each decision period, the agents jointly determine replenishment quantities and lateral transshipment quantities. The purpose is to reduce inventory operating cost while keeping transfers feasible and avoiding policies that repeatedly expose the same donor site to shortage risk.

The simulator is part of the contribution, not only an implementation detail. It defines a common test environment for policies that must make replenishment and lateral transshipment decisions under the same demand process, transition rules, feasibility checks, and cost accounting. It is useful because closed-form analysis is difficult when demand is uncertain, sites have different local states, decisions are coupled through transfers, and a local action can change the shortage risk of another site. The notation used in this section is summarized in Table II, and the simulator design is summarized in Table III.

The scenario is shown at a high level in Fig. 1. The figure is meant to make the benchmark structure clear: the simulator provides site states, forecasts, and feedback; local inventory agents choose replenishment and transshipment actions; and the proposed method generates the executable decision logic that is tested in the simulator.

This figure also marks the boundary of the paper. The work focuses on policy generation and evaluation for the replenish-and-transship simulator. It does not claim to model every supply-chain process, such as procurement contracts, routing,

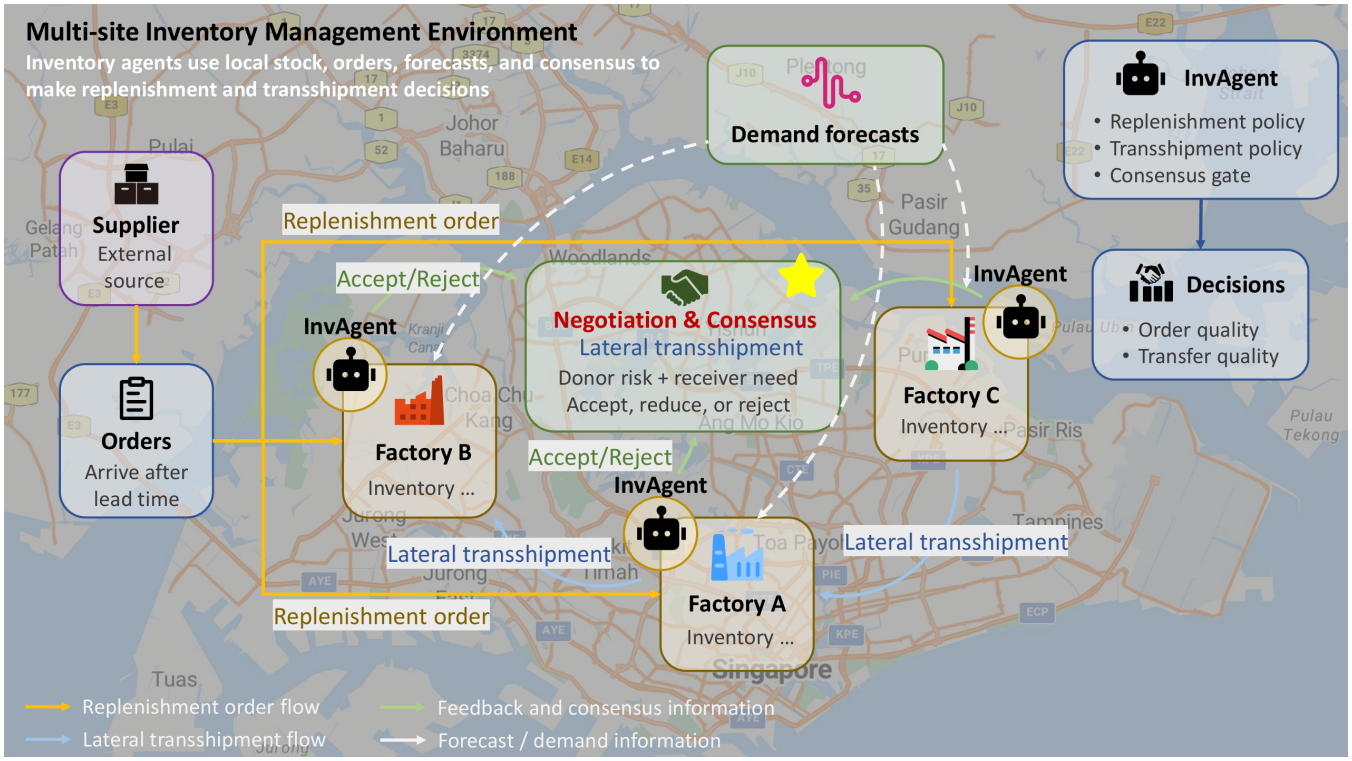


Fig. 1. Illustrative multi-site inventory environment using Singapore as an example geography. Each site represents a local factory, warehouse, or inventory location operated by an inventory-control agent that observes local inventory state and demand forecasts. The simulator updates demand, replenishment lead times, lateral transshipment, feasibility checks, and cost accounting. The proposed method generates executable policy logic for these agents, and candidate policies are evaluated using cost, service, and coordination measures.

pricing, or production scheduling.

A. System State and Demand

We consider a finite-horizon periodic-review system with N sites and decision periods $t = 1, \ldots, T$. The current simulator supports the two-site and three-site cases. At the beginning of a decision period, the policy observes a state $S(t)$ containing local site information and pairwise transshipment information. For each site, the local information includes current inventory, holding cost, lost-sales cost, storage capacity, production-capacity parameter, fixed and unit ordering costs, pipeline order information, a demand forecast vector, and transshipment cost parameters. The forecast vector is written as

$$\hat{\mathbf{d}}_{i,t} = \left[\hat{d}_{i,t}^{(1)}, \hat{d}_{i,t}^{(2)}, \ldots, \hat{d}_{i,t}^{(K)}\right], \tag{1}$$

where K is the number of forecast features exposed to the policy. The general formulation allows any K. The current experiments use $K = 2$, matching the two forecast values exposed by the implemented simulator state.

For synthetic scenarios, the simulator first samples a time-varying demand mean for each site and period. The realized demand $D_i(t)$ is then sampled from a Poisson distribution with that period-specific mean. The forecast features are generated from the demand model and provide short-horizon demand information to the policy. For test scenarios based on historical demand records, the simulator uses the provided demand sequence and uses the same sequence as the forecast record. In both cases, future realized demand is not known when the policy chooses the action.

B. Decision Variables and Action Structure

At each period, the policy chooses a replenishment vector and a transshipment vector. The replenishment vector is

$$\mathbf{x}(t) = [x_1(t), \ldots, x_N(t)], \tag{2}$$

where $x_i(t)$ is the order quantity for site i. Orders are not available immediately. They enter the pipeline and arrive after the replenishment lead time L_o.

The transshipment vector contains one signed quantity for each unordered site pair. For a pair (i, j) with $i < j$, $q_{ij}(t) > 0$ denotes a transfer from site i to site j, while $q_{ij}(t) < 0$ denotes a transfer from site j to site i. For two sites, the action contains one transshipment quantity. For three sites, the action contains quantities for pairs $(1, 2)$, $(1, 3)$, and $(2, 3)$.

The complete action is therefore

$$\mathbf{d}(t) = (\mathbf{q}(t), \mathbf{x}(t)), \tag{3}$$

where the transshipment part appears before the replenishment part in the simulator action vector.

C. Inventory Transition

For each site, an outstanding order arrives at the start of the simulator step. The inventory after order arrival is capped by storage capacity:

$$\bar{I}_i(t) = \min\{Cap_i, I_i(t) + A_i(t)\}, \tag{4}$$

TABLE II
NOTATION

Symbol	Description
N	Number of inventory sites
T	Number of decision periods in one simulation episode
i, j	Site indices, with $i, j \in \{1, \dots, N\}$
$D_i(t)$	Random demand at site i in period t
$\hat{d}_{i,t}^{(\kappa)}$	The κth demand forecast feature for site i at period t
K	Number of demand forecast features exposed to the policy
L_o	Replenishment lead time
$I_i(t)$	On-hand inventory at site i at the start of period t
$P_i(t)$	Pipeline order information available at site i
Cap_i	Storage capacity of site i
U_i	Production-capacity parameter exposed to the policy state
$x_i(t)$	Replenishment quantity ordered for site i in period t
$q_{ij}(t)$	Lateral transshipment quantity between sites i and j
a_i	Per-unit ordering cost at site i
k_i	Fixed ordering cost at site i
h_i	Holding cost per positive inventory unit at site i
b_i	Lost-sales cost per shortage unit at site i
c^{tr}	Per-unit transshipment cost
K^{tr}	Fixed cost charged when a transshipment is made
$S(t)$	State observed by the policy at period t
C_t	Total cost incurred in period t
π	Inventory policy used to choose replenishment and transshipment actions

TABLE III
SIMULATOR BENCHMARK OVERVIEW

Component	Benchmark specification
Inputs	Number of sites, episode length, initial inventory, capacity, production-capacity parameter, ordering costs, holding costs, lost-sales costs, transshipment costs, demand model, and forecast features
State	Site-level inventory, costs, capacity, pipeline order information, forecast vector, and pairwise transshipment states
Actions	Pairwise signed transshipment quantities and site-level replenishment quantities
Demand	Synthetic stochastic demand or loaded demand records; synthetic demand uses time-varying Poisson means
Transition	Order arrival, demand realization, lateral transshipment, cost calculation, pipeline update, and forecast update
Costs	Transshipment cost, holding and lost-sales cost, and order cost
Evaluation	Average cost per episode plus service and coordination metrics, including stockout proxy, accepted/rejected transfers, and fairness debt
Configurability	Site number, demand scale, cost asymmetry, capacity, initial inventory, lead time, episode length, and benchmark scenario

where $A_i(t)$ is the pipeline order that arrives in period t. Demand is then realized and subtracted:

$$\tilde{I}_i(t) = \bar{I}_i(t) - D_i(t). \tag{5}$$

After this demand update, lateral transshipments are applied. A proposed transfer is executed only if the donor has enough available inventory at the point where the transfer is checked. If the donor inventory is insufficient, the simulator sets that transfer to zero. Let $Q_i(t)$ denote the net inventory change at site i caused by all accepted transshipments in period t. The inventory after transshipment is

$$I_i^+(t) = \tilde{I}_i(t) + Q_i(t). \tag{6}$$

If $I_i^+(t)$ is positive, the site pays holding cost. If $I_i^+(t)$ is negative, the shortage is charged as lost sales and the inventory carried to the next period is reset to zero:

$$I_i(t+1) = \max\{0, I_i^+(t)\}. \tag{7}$$

The new replenishment decision $x_i(t)$ is then stored in the pipeline and becomes available after the lead time.

D. Feasibility Constraints

The simulator enforces simple feasibility rules. Replenishment quantities are nonnegative after policy adaptation and are capped before entering the pipeline. In the current simulator step, any replenishment quantity above site capacity is reduced to Cap_i:

$$0 \le x_i(t) \le Cap_i. \tag{8}$$

For LLM-generated policies, the adapter first clips order quantities to a nonnegative range before the simulator applies its own capacity cap.

Transshipment feasibility is checked through donor availability. For each proposed pairwise transfer, the donor must have enough inventory to cover the requested movement after prior transfers in the same period have been considered. Otherwise, that pairwise movement is not executed. This rule prevents the simulator from creating negative donor inventory through transshipment alone, although a site may still have negative inventory after demand if it faces shortage before receiving enough transfer support.

E. Cost and Objective

The one-period cost contains three parts: transshipment cost, holding and lost-sales cost, and order cost. The transshipment cost is

$$C_t^{tr} = \sum_{(i,j)} \left(c^{tr} |q_{ij}^{acc}(t)| + K^{tr} \mathbb{I}\{|q_{ij}^{acc}(t)| > 0\} \right), \tag{9}$$

where $q_{ij}^{acc}(t)$ is the accepted transfer after feasibility checking. The holding and lost-sales cost is

$$C_t^{hl} = \sum_{i=1}^{N} \left(h_i \max\{I_i^+(t), 0\} + b_i \max\{-I_i^+(t), 0\} \right). \tag{10}$$

The order cost is

$$C_t^{ord} = \sum_{i=1}^{N} \left(a_i x_i(t) + k_i \mathbb{I}\{x_i(t) > 0\} \right). \tag{11}$$

The total period cost is

$$C_t = C_t^{tr} + C_t^{hl} + C_t^{ord}. \tag{12}$$

Given a policy π, the simulator evaluates the average cost over one episode:

$$J_T(\pi) = \frac{1}{T} \sum_{t=1}^{T} C_t. \tag{13}$$

The optimization goal is to find a policy with low expected average cost across demand realizations and test scenarios:

$$\min_{\pi} \ \mathbb{E}\left[J_T(\pi)\right]. \tag{14}$$

The evaluation also records service and coordination measures, including stockout proxy count, accepted transfer count, rejected transfer count, and fairness debt. Fairness debt is computed in the current prototype as the sum of absolute net pairwise transfers over the episode. These measures do not replace the cost objective. They are used to judge whether a low-cost policy achieves its performance through reasonable coordination behavior.

F. Benchmark Scope

The benchmark is designed for researchers who need a repeatable environment for multi-site, multi-agent inventory control with lateral transshipment. It differs from a single-site inventory benchmark because decisions at one site can change the shortage risk at another site. It differs from a standard multi-echelon benchmark because the sites considered here are at the same echelon and can share inventory laterally. It also differs from a pure ordering benchmark because a policy must decide both replenishment and pairwise transfer quantities.

The simulator can be used to compare hand-designed rules, GP policies, DRL policies, LLM-generated code policies, and other policy-search methods under the same demand and cost setting. Researchers can configure the number of sites supported by the simulator, demand scale, cost asymmetry, initial inventory, capacity, lead time, and episode length. The present implementation supports two-site and three-site replenish-and-transship scenarios; extending the simulator to larger networks is a natural next step.

IV. Proposed Method

A. Framework Overview

ConsensusCode-Evolve is an offline generate–validate–simulate–select loop. The LLM is used only during design time. It receives a compact summary of previous policy performance and writes new candidate policy code. Each candidate is checked by a static safety validator before it is executed. Valid candidates are then evaluated in the inventory simulator. The best policies and their metrics are archived and used to construct the next prompt.

Figure 2 summarizes the method pipeline. The figure separates the design-time search loop from the deployment-time controller. During search, candidate policy programs are generated, checked, evaluated, and selected. During deployment, only the selected executable policy is used.

The pipeline is intentionally simple. The LLM proposes code, but it does not judge the final policy. The simulator and the safety checker determine which candidates can enter the archive.

The method has four design principles. First, the deployed controller must not depend on an LLM call. After search, the selected policy is ordinary Python code. Second, the policy representation must separate local replenishment, transfer proposal, and transfer agreement. This separation makes the transshipment logic easier to inspect. Third, the simulator, not the LLM, decides whether a policy is good. Fourth, generated code must be restricted to a small deterministic subset of Python.

[AUTHOR CHECK: export the editable HTML source to PDF/SVG/PNG and include the rendered figure here.]
figures/proposed_method_pipeline.html

Fig. 2. Proposed ConsensusCode-Evolve pipeline. Site states and K forecast features are converted into a policy-code interface. An offline LLM mutation loop proposes candidate replenishment, transshipment, and consensus-gate functions. Valid code is evaluated in the inventory simulator, selected through an archive, and deployed as executable decision logic without an online LLM call.

B. State Representation

The current implementation converts simulator arrays into dictionaries before calling the policy code. In the general formulation, each site state contains the site index, inventory level, holding cost, lost-sales cost, capacity, production-capacity parameter, fixed and unit ordering costs, pipeline inventory, a forecast vector $\hat{\mathbf{d}}_{i,t}$ with K entries, and transshipment cost parameters. For transshipment, the state of a source-target pair contains the same local information for both sites. The global state contains the current period, episode length, number of sites, total inventory, total forecast over the exposed forecast vector, average inventory, inventory imbalance, and forecast imbalance. The implemented experiments use $K = 2$ and expose these entries as `forecast_1` and `forecast_2` in the policy dictionaries.

This dictionary representation is used for two reasons. It gives generated code meaningful variable names, which improves readability. It also hides simulator internals from the LLM. The LLM does not receive direct access to the environment object, files, random generators, or external libraries.

C. Policy-Code Representation

ConsensusCode-Evolve represents an inventory policy as three constrained Python functions:

- `replenishment_policy(site_state, global_state)` returns a nonnegative order quantity for one site.
- `transshipment_proposal(source_state, target_state, global_state)` returns a signed or directional proposed transfer quantity.
- `consensus_gate(source_state, target_state, proposed_quantity, history)` returns the accepted transfer quantity after agreement filtering.

The first function is called once for each site and returns the replenishment quantity. The second function is called for a pair of sites and returns a proposed signed transfer. In the implementation, a nonnegative proposal means the first site is the donor and the second site is the receiver; a negative proposal reverses the direction. The third function receives the selected source, target, proposed quantity, and transfer

history. It returns the accepted quantity. The adapter clamps the accepted quantity to the interval between zero and the proposed quantity and rounds the final action before passing it to the simulator.

This representation differs from a direct action policy. A direct policy would output an order and transfer vector in one step. Here, transshipment is split into proposal and agreement. This is useful because an LLM-generated proposal may identify an imbalance, while the consensus gate can still reject or reduce the transfer if the donor is unsafe or if pairwise transfer history is unbalanced.

D. Consensus-Seeking Transshipment

For each potential pairwise transfer, the framework first computes a proposed quantity and then applies a consensus gate. The gate is interpreted as a local agreement mechanism. A typical hand-designed form is

$$q_{ij,t}^{\mathrm{acc}} = q_{ij,t}^{\mathrm{prop}} \cdot w_{i,t}^{\mathrm{donor}} \cdot u_{j,t}^{\mathrm{receiver}} \cdot f_{ij,t}^{\mathrm{fair}}, \tag{15}$$

where $q_{ij,t}^{\mathrm{prop}}$ is the proposed transfer from site i to site j, $w_{i,t}^{\mathrm{donor}}$ measures donor willingness, $u_{j,t}^{\mathrm{receiver}}$ measures receiver urgency, and $f_{ij,t}^{\mathrm{fair}}$ penalizes repeated one-way support. These terms may be hand-designed in seed policies or generated directly by the LLM as executable code.

Donor willingness can depend on the donor's inventory, forecast, capacity, and shortage cost. Receiver urgency can depend on low inventory, high forecast, and high lost-sales cost. The fairness term can use transfer history to reduce repeated support in the same direction. The current prototype stores the net transfer for each site pair, the accepted transfer count, and the rejected transfer count. These variables are simple, but they make the agreement logic observable.

This structure is important for industrial automation because it prevents the GAI system from generating opaque transfer commands. Each accepted movement can be inspected in terms of donor safety, receiver need, and transfer history. The paper does not claim that this gate solves general multi-agent negotiation. It is a lightweight consensus layer for inventory transshipment.

E. Safe Generative Code Search

The LLM is used only between simulation batches. In each round, the algorithm provides a compact prompt containing the allowed API, available state keys, safety rules, the scenario name and seed, the current top policy scores, and selected policy source code. The score table includes fitness, transshipment cost, holding/lost-sales cost, order cost, stockout proxy count, accepted transfer count, rejected transfer count, and fairness debt. The LLM is asked to generate a small fixed number of deterministic candidate policy files. The current prototype asks for three candidates per round.

Before simulation, each candidate is checked by an abstract-syntax-tree validator. The current safe subset allows only top-level definitions of the three required functions. It permits assignments, returns, conditionals, arithmetic, Boolean expressions, comparisons, dictionary indexing, constants, and calls to `abs`, `min`, `max`, and `round`. It disallows imports, file operations, network access, random calls, classes, decorators, attributes, list or dictionary construction, unexpected functions, and dunder names. Candidates that fail validation are rejected without execution.

Algorithm 1 Offline ConsensusCode-Evolve search loop.

```
Initialize archive A with seed policy files.
Validate each seed policy with the AST checker.
Evaluate each valid seed policy in the simulator.
Store source code and metrics in A.
for round r = 1, 2, ..., R do
    Select the top k archived policies by fitness.
    Build a compact prompt with API rules, state keys, metrics, and selected source code.
    Ask the LLM to generate m candidate policy files.
    for each generated candidate p do
        Run AST validation on p.
        if p is valid then
            Evaluate p in the inventory simulator.
            Add source code and metrics to A.
        else
            Record the validation error and reject p.
        end if
    end for
end for
Return the selected archived policy for deployment or further testing.
```

The safety checker is not a full sandbox. It is a first filter that prevents common unsafe or non-reproducible generated code patterns before simulator evaluation. [AUTHOR CHECK: describe any additional runtime isolation used when experiments are run, or state that AST validation is the current safety layer.]

F. Evolution Loop

The search loop starts from human-written seed policies. These seeds are not expected to be optimal. Their purpose is to provide the LLM with valid examples of the API and with initial behavior for comparison. After evaluation, valid candidates are sorted by fitness and stored with source code and metrics. The archive is then used to build the next mutation prompt.

In the current implementation, $k = 3$ top policies are included in the prompt and $m = 3$ new candidates are requested. The archive is stored as JSON and CSV files under the research output directory. Generated candidates can be supplied either as separate Python files or as fenced Python blocks in a response file, which the mutation loop extracts before validation.

The evaluator, rather than the LLM, determines whether a candidate is useful. This is a key difference from direct LLM planning. The LLM may suggest an action rule, but only simulator performance and validation decide whether the rule enters the archive.

G. Training, Inference, and Deployment

The method has a design-time phase and a deployment phase. During design time, the LLM generates policy-code

candidates, and the simulator evaluates them. This phase may use several scenarios, random seeds, and stress tests. During deployment, the selected policy is executed directly as Python code. No LLM call is needed to choose orders or transfers.

At each deployment decision period, the simulator or industrial control system builds the state dictionaries, calls `transshipment_proposal` and `consensus_gate` for candidate transfer pairs, then calls `replenishment_policy` for each site. The resulting action vector is passed to the inventory system. Because the policy is small code, an engineer can inspect thresholds, arithmetic expressions, and conditionals before use. [AUTHOR CHECK: specify whether final deployment will use all pairwise transfers, nearest-neighbor transfers, or another network topology in the industrial case.]

H. Differences from Existing Methods

ConsensusCode-Evolve differs from the method categories reviewed in Section II in several concrete ways. Compared with classical inventory rules, it searches over generated policy code rather than fixing thresholds by hand. Compared with GP, it asks an LLM to make semantic code edits under an API, while still relying on simulation for selection. Compared with RL, it produces a small inspectable policy program instead of a neural controller. Compared with online LLM agents, it removes the LLM from the control loop and uses it only as an offline generator. Compared with ordinary LLM code generation, it evaluates candidates in an inventory simulator and includes a consensus gate tailored to transshipment.

V. Experimental Design

A. Research Questions

The experiments are designed to answer four questions. RQ1 asks whether LLM-evolved policy code can reduce average inventory cost compared with rule-based, GP-based, and learning-based baselines under the same simulator. RQ2 asks whether the consensus gate changes transshipment behavior in a useful way, measured by accepted transfers, rejected transfers, fairness debt, and stockout proxy counts. RQ3 asks whether the method uses LLM calls efficiently, measured by the number of generated candidates, token budget, and valid-policy rate. RQ4 asks whether the selected policies remain stable under changes in demand level, holding and lost-sales costs, and network size.

B. Scenarios and Data

The current implementation evaluates policies in the multi-site replenishment and transshipment simulator used by the inventory-control codebase. The available scenario interface is `ScenarioDesign_replenish_transship`. The initial prototype has been exercised on the synthetic scenario `sN2h_1_5b2` with seed 0. The formulation allows any number K of forecast features, while the current experimental implementation uses $K = 2$. [AUTHOR CHECK: list all final datasets, industrial cases, synthetic scenarios, random seeds, episode lengths, number of sites, demand settings, forecast features, lead times, capacity settings, and cost parameters.]

The scenario set includes at least two-site and three-site cases because the consensus mechanism is most meaningful when more than one donor-receiver relationship can occur. Stress scenarios vary demand imbalance, cost asymmetry, and capacity tightness. [AUTHOR CHECK: confirm the scenario set and specify whether Teckwah, Sanwa, Taiyo, or other industrial cases are included. Describe the data source.]

C. Compared Methods

The main comparison includes the following methods when available:

- Classical or rule-based inventory policies, such as reorder-up-to or (s, S) baselines. [AUTHOR CHECK: define the exact rule and parameter tuning method.]
- Hand-designed consensus policies using the same three-function API.
- The existing multi-tree GP replenishment-transshipment method, in which separate GP trees produce replenishment and transshipment decisions.
- Learning-based baselines, such as PPO or another DRL policy already implemented in the repository. [AUTHOR CHECK: state whether the DRL baseline is trained for the same action space and scenarios.]
- ConsensusCode-Evolve without the consensus gate, where proposed transshipments are applied directly after feasibility clipping.
- The full ConsensusCode-Evolve method with replenishment policy, transshipment proposal, and consensus gate.

This set separates three effects: the value of generated code, the value of simulator-grounded search, and the value of consensus gating.

D. Evaluation Metrics

The primary metric is average total cost per period. The cost is decomposed into transshipment cost, holding and lost-sales cost, and order cost. Service behavior is measured by stockout frequency or the current stockout proxy count. Coordination behavior is measured by accepted transfer count, rejected transfer count, transshipment frequency, and fairness debt. Search efficiency is measured by valid-policy rate, number of evaluated candidates, number of LLM calls, and token usage. [AUTHOR CHECK: provide the exact token-counting method and whether prompt and completion tokens are counted separately.]

The reported results use means and standard deviations over random seeds. Comparisons among stochastic methods are tested statistically. [AUTHOR CHECK: choose the final statistical test, such as Wilcoxon signed-rank test or paired t-test, and state the significance level.]

E. Parameter Settings and Implementation

In the current prototype, the archive prompt includes the top $k = 3$ policies, and each LLM call asks for $m = 3$ new candidate files. Candidate code must pass the AST validator

before execution. The allowed built-in functions are `abs`, `min`, `max`, and `round`. The policy adapter clips replenishment quantities to a nonnegative range and clips accepted transshipment quantities to the proposed amount.

[AUTHOR CHECK: report the final LLM model, decoding settings, maximum tokens, prompt template version, number of mutation rounds, number of candidates per round, CPU/GPU hardware, operating system, Python version, DEAP version, and simulator version or commit hash.]

F. Ablation and Sensitivity Studies

The ablation study tests whether each design choice changes behavior. The minimum ablations are: no consensus gate, no fairness history, no donor-protection term, no receiver-urgency term, and different archive sizes. A separate sensitivity study varies token budget, number of generated candidates, demand variance, cost asymmetry, and network size. [AUTHOR CHECK: provide the final values used for each ablation and sensitivity factor.]

VI. Experimental Results

A. Main Comparison

[AUTHOR CHECK: Insert the main comparison table with average total cost, cost components, service metrics, and coordination metrics for all baselines and ConsensusCode-Evolve.]

This comparison addresses RQ1. The analysis identifies which method obtains the lowest average cost, whether the difference is consistent across seeds, and which cost component explains the difference. If ConsensusCode-Evolve improves total cost, the discussion separates gains from replenishment and gains from transshipment. If it does not improve cost, the result is reported directly, together with any observed change in interpretability, valid-policy rate, or coordination behavior.

B. Consensus-Gate Analysis

[AUTHOR CHECK: Insert ablation results comparing direct transshipment, proposal-only policy, consensus gate without fairness history, and full consensus gate.]

This analysis addresses RQ2 by testing whether the consensus gate reduces harmful transfers without increasing stockouts. Evidence for improved coordination may appear as fewer repeated one-way transfers, lower fairness debt, or fewer donor shortages. Such a result would support the claim that explicit agreement logic changes coordination behavior. It would not by itself prove that the method is optimal or that the same gate works in every network.

C. Search Efficiency and Safety

[AUTHOR CHECK: Insert the number of generated candidates, valid candidates, rejected candidates, LLM calls, token usage, and best fitness by round.]

This analysis addresses RQ3 by measuring whether compact archive feedback helps the LLM produce valid and useful policy code over rounds. The valid-policy rate matters because unsafe or invalid code reduces the value of generative search. Token usage matters because the method is motivated by offline, low-frequency LLM use rather than online LLM control.

D. Sensitivity and Stress Tests

[AUTHOR CHECK: Insert sensitivity results for demand variance, cost asymmetry, capacity tightness, network size, and random seed.]

This analysis addresses RQ4 by testing whether policies found in one setting remain useful when the environment changes. If performance drops under a stress case, the failure is described. Examples include over-ordering when demand is low, refusing transfers too often when lost-sales costs are high, or draining one donor in a three-site network. Such failure cases are useful because they show where the policy representation or archive feedback needs improvement.

E. Qualitative Policy Inspection

[AUTHOR CHECK: Insert one or two selected generated policies or simplified code excerpts, together with a short explanation of the decision logic.]

This analysis shows whether the final policy can be inspected by engineers. The discussion identifies the main thresholds or expressions used by the generated code and relates them to inventory concepts such as safety stock, donor risk, receiver urgency, and transfer history. The purpose is not to claim that the code is always simple, but to test whether the generated policy is more transparent than a neural controller.

F. Summary of Evidence

[AUTHOR CHECK: Complete this paragraph after the experimental tables are available. State which research questions are supported, which are only partly supported, and which remain open.]

The results should be interpreted conservatively. Lower cost in a simulator supports the usefulness of the generated policy for the tested scenarios. It does not prove plant-wide optimality, safety in unseen industrial settings, or superiority for all inventory networks. Those claims require broader validation.

References

[1] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. Mankowitz, E. Sutherland Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[2] H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi, "CodeRL: Mastering code generation through pretrained models and deep reinforcement learning," in *Advances in Neural Information Processing Systems*, 2022.

[3] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, 2023.

[4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, 2023.

[5] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.

[6] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, "Eureka: Human-level reward design via coding large language models," in *International Conference on Learning Representations*, 2024.

[7] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, pp. 468–475, 2024.

[8] P. H. Zipkin, *Foundations of Inventory Management*. McGraw-Hill, 2000.

[9] S. Axsater, *Inventory Control*, 3rd ed. Springer, 2015.

[10] C. Paterson, G. Kiesmuller, R. Teunter, and K. Glazebrook, "Inventory models with lateral transshipments: A review," *European Journal of Operational Research*, vol. 210, no. 2, pp. 125–136, 2011.

[11] K. Kogan, S. Lou, C. S. Tapiero, and M. Shnaiderman, "Supply chain with inventory review and dependent demand distributions: Dynamic inventory outsourcing," *IEEE Transactions on Automation Science and Engineering*, vol. 7, no. 2, pp. 197–207, 2010.

[12] M. Boccadoro, F. Martinelli, and P. Valigi, "Supply chain management by H-infinity control," *IEEE Transactions on Automation Science and Engineering*, vol. 5, no. 4, pp. 703–707, 2008.

[13] A. Castelnuovo, L. Ferrarini, and L. Piroddi, "An incremental petri net-based approach to the modeling of production sequences in manufacturing systems," *IEEE Transactions on Automation Science and Engineering*, vol. 4, no. 3, pp. 424–434, 2007.

[14] L. Liu, C.-B. Yan, and J. Li, "Modeling, analysis, and improvement of batch-discrete manufacturing systems: A systems approach," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 3, pp. 1567–1585, 2022.

[15] C. Zhang, Q. Gao, M. V. Basin, J. Lu, and H. Liu, "Robust control of multi-line re-entrant manufacturing plants via stochastic continuum models," *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 4, pp. 4923–4935, 2024.

[16] I.-B. Park, J. Huh, J. Kim, and J. Park, "A reinforcement learning approach to robust scheduling of semiconductor manufacturing facilities," *IEEE Transactions on Automation Science and Engineering*, pp. 1–12, 2020.

[17] S. Luo, L. Zhang, and Y. Fan, "Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 4, pp. 3020–3038, 2022.

[18] X. Wang, Y. Laili, L. Zhang, and Y. Liu, "Hybrid task scheduling in cloud manufacturing with sparse-reward deep reinforcement learning," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 1878–1892, 2025.

[19] M. C. Fu, "Simulation optimization," in *Handbook of Simulation Optimization*. Springer, 2015.

[20] S. Amaran, N. V. Sahinidis, B. Sharda, and S. J. Bury, "Simulation optimization: A review of algorithms and applications," *Annals of Operations Research*, vol. 240, no. 1, pp. 351–380, 2016.

[21] X. Shen, S. Liu, B. Zhou, T. Wu, Q. Zhang, and J. Bao, "Digital twin-driven reinforcement learning method for marine equipment vehicles scheduling problem," *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 3, pp. 2173–2183, 2024.

[22] C.-Y. Lin, T.-L. Tseng, and T.-H. Tsai, "A digital twin framework with deep feature extraction and gaussian process for multi-objective optimization in semiconductor manufacturing," *IEEE Transactions on Automation Science and Engineering*, vol. 23, pp. 1928–1949, 2026.

[23] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.

[24] R. Poli, W. B. Langdon, and N. F. McPhee, *A Field Guide to Genetic Programming*. Lulu, 2008.

[25] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[26] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[27] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[28] R. Olfati-Saber, J. A. Fax, and R. M. Murray, "Consensus and cooperation in networked multi-agent systems," *Proceedings of the IEEE*, vol. 95, no. 1, pp. 215–233, 2007.

[29] C. Wang, "Requirement-based bidding language for agent-based scheduling," *IEEE Transactions on Automation Science and Engineering*, vol. 10, no. 1, pp. 219–223, 2013.

[30] W. Wei, C. Jiang, and Y. Huang, "A data-driven human-machine collaborative product design system toward intelligent manufacturing," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 736–749, 2025.

[31] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," *arXiv preprint arXiv:2309.03409*, 2023.

[32] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, "Promptbreeder: Self-referential self-improvement via prompt evolution," *arXiv preprint arXiv:2309.16797*, 2023.

[33] S. K. Chaharsooghi, J. Heydari, and S. H. Zegordi, "A reinforcement learning model for supply chain ordering management: An application to the beer game," *Decision Support Systems*, vol. 45, no. 4, pp. 949–959, 2008.

[34] C. D. Hubbs, H. D. Perez, O. Sarwar, N. V. Sahinidis, I. E. Grossmann, and J. M. Wassick, "OR-gym: A reinforcement learning library for operations research problems," *arXiv preprint arXiv:2008.06319*, 2020.

[35] X. Liu, M. Hu, Y. Peng, and Y. Yang, "Multi-agent deep reinforcement learning for multi-echelon inventory management," *Production and Operations Management*, vol. 34, no. 7, pp. 1836–1856, 2024.

[36] J. Chen, "Multi-location inventory management under time-varying demands: A deep reinforcement learning approach," in *Proceedings of the International Conference on Industrial Engineering and Operations Management*, 2024.

[37] A. J. Clark and H. Scarf, "Optimal policies for a multi-echelon inventory problem," *Management Science*, vol. 6, no. 4, pp. 475–490, 1960.

[38] D. Bertsimas and A. Thiele, "A robust optimization approach to inventory theory," *Operations Research*, vol. 54, no. 1, pp. 150–168, 2006.

[39] A. A. Syntetos, M. Z. Babai, J. E. Boylan, S. Kolassa, and K. Nikolopoulos, "Supply chain forecasting: Theory, practice, their gap and the future," *European Journal of Operational Research*, vol. 252, no. 1, pp. 1–26, 2016.